Лабораторная работа №3
Базовые растровые алгоритмы

Лепеев Вадим

11 ноября 2026 г.

1 Цель работы

Практическое освоение и реализация базовых алгоритмов растеризации (дискретизации) отрезков и кривых: пошаговый алгоритм, алгоритм ЦДА, алгоритмы Брезенхема (для линии и окружности). Дополнительно реализован алгоритм сглаживания Сяолиня Ву (Xiaolin Wu). Задачей работы является не только визуализация, но и строгое разделение логики вычислений и графического интерфейса для корректного замера времени работы алгоритмов.

2 Математика алгоритмов

Для отображения математических фигур на дискретной сетке пикселей необходимо определить, центры каких клеток лежат ближе всего к идеальной геометрической линии.

2.1 Пошаговый алгоритм (наивный подход)

Основан на прямом использовании уравнения прямой $y = kx + b$, где $k = \frac{\Delta y}{\Delta x}$, а $b = y_1 - kx_1$. Для каждого целого x_i от x_1 до x_2 вычисляется точное значение ординаты, которое затем округляется до ближайшего целого:

$$y_i = \lfloor kx_i + b + 0.5 \rfloor$$

Ограничение: если $|\Delta y| > |\Delta x|$ (угол наклона $> 45°$), шаг по x приведёт к разрывам линии. В этом случае роли осей принудительно меняются местами: шаг идёт по y, а x вычисляется как $x = \frac{1}{k}y - \frac{b}{k}$. Метод требует операций умножения и округления `float` на каждом шаге.

2.2 Алгоритм ЦДА (Цифровой дифференциальный анализатор)

Вместо вычисления функции с нуля на каждом шаге, алгоритм использует приращения (интегрирование). Отрезок задаётся параметрически, где длина пути $L = \max(|\Delta x|, |\Delta y|)$. На каждом шаге координаты получают фиксированное дробное приращение:

$$X_{inc} = \frac{\Delta x}{L}, \quad Y_{inc} = \frac{\Delta y}{L}$$

Тогда следующие координаты вычисляются рекурсивно: $x_{i+1} = x_i + X_{inc}$ и $y_{i+1} = y_i + Y_{inc}$. Математически умножение заменяется на сложение, однако алгоритм всё ещё работает с вещественными числами и накапливает ошибку (погрешность машинного округления).

2.3 Алгоритм Брезенхема (Отрезок)

Полностью исключает числа с плавающей точкой. Пусть текущий пиксель (x_i, y_i). Кандидатами на следующий пиксель (при движении вдоль оси X) выступают $(x_i + 1, y_i)$ и $(x_i + 1, y_i + 1)$. Разность расстояний от этих центров до идеальной линии $\Delta d = d - d$. Умножение этой разности на Δx даёт *целочисленный параметр ошибки* e_i. Начальное значение $e_0 = 2\Delta y - \Delta x$. Рекуррентное соотношение:

$$e_{i+1} = \begin{cases} e_i + 2\Delta y - 2\Delta x, & \text{если } e_i \geq 0 \quad \text{(шаг по Y совершается)} \\ e_i + 2\Delta y, & \text{если } e_i < 0 \quad \text{(шаг по Y не совершается)} \end{cases}$$

Алгоритм требует только сложения целых чисел и битовых сдвигов (умножение на 2).

2.4 Алгоритм Брезенхема (Окружность)

Уравнение окружности $x^2 + y^2 = R^2$. Используется 8-сторонняя симметрия: вычисляются точки только для 1-го октанта (от $x = 0$ до $x = y$, угол $0-45°$), а остальные 7 точек получаются зеркальными отражениями координат $(\pm x, \pm y)$ и $(\pm y, \pm x)$. Параметр принятия решения d_i определяет, находится ли дуга ближе к пикселю (x_i+1, y_i) или $(x_i + 1, y_i - 1)$. Начальное значение $d_0 = 3 - 2R$.

$$d_{i+1} = \begin{cases} d_i + 4x + 6, & \text{если } d_i < 0 \quad (y \text{ не меняется}) \\ d_i + 4(x - y) + 10, & \text{если } d_i \geq 0 \quad (y \text{ уменьшается на } 1) \end{cases}$$

Позволяет рисовать окружности без тригонометрических функций и извлечения корня.

2.5 Сглаживание по Ву (Бонус)

Для устранения визуальных «ступенек» (алиасинга) идеальная линия распределяет свою яркость между двумя соседними клетками. Если идеальная ордината $y^* = kx + b$, то алгоритм определяет базовый пиксель $y_{int} = \lfloor y^* \rfloor$ и вычисляет дробную часть $f = y^* - y_{int}$. Яркость (интенсивность цвета) распределяется пропорционально:

$$I_{\text{верхнего}} = f, \quad I_{\text{нижнего}} = 1 - f$$

Сумма яркостей в столбце всегда равна 1. Этот метод возвращается к `float`-арифметике, но обеспечивает визуальное субпиксельное сглаживание за счёт манипуляции прозрачностью.

3 Сопроводительные вычисления (Ручная трассировка)

Рассмотрим работу алгоритма Брезенхема для отрезка $(0,0) \to (4,3)$. Имеем: $\Delta x = 4$, $\Delta y = 3$. Начальная ошибка $err = \Delta x - \Delta y = 1$. Сравнивается величина $e2 = 2 \cdot err$.

X	**Y**	**Err**	**Условие (e2)**	**Действие**
0	0	1	$2 > -3$ и $2 < 4$	$X+1, Y+1$, $err = 1-3+4 = 2$
1	1	2	$4 > -3$	$X+1$, $err = 2-3 = -1$
2	1	-1	$-2 < 4$	$Y+1$, $err = -1+4 = 3$
2	2	3	$6 > -3$	$X+1$, $err = 3-3 = 0$
3	2	0	$0 > -3$ и $0 < 4$	$X+1, Y+1$, $err = 0-3+4 = 1$
4	3	-	Конец	-

Таблица 1: Трассировка алгоритма Брезенхема

4 Особенности реализации и интерфейс

Графический интерфейс разработан с использованием библиотеки `CustomTkinter`. Реализовано интерактивное масштабирование (от 2 до 50 пикселей на клетку), скрытие сетки при мелком масштабе для оптимизации рендеринга и возможность задавать координаты кликами ЛКМ по холсту. Вывод журнала вычислений осуществляется отдельным прогоном алгоритма, чтобы не искажать замеры времени.

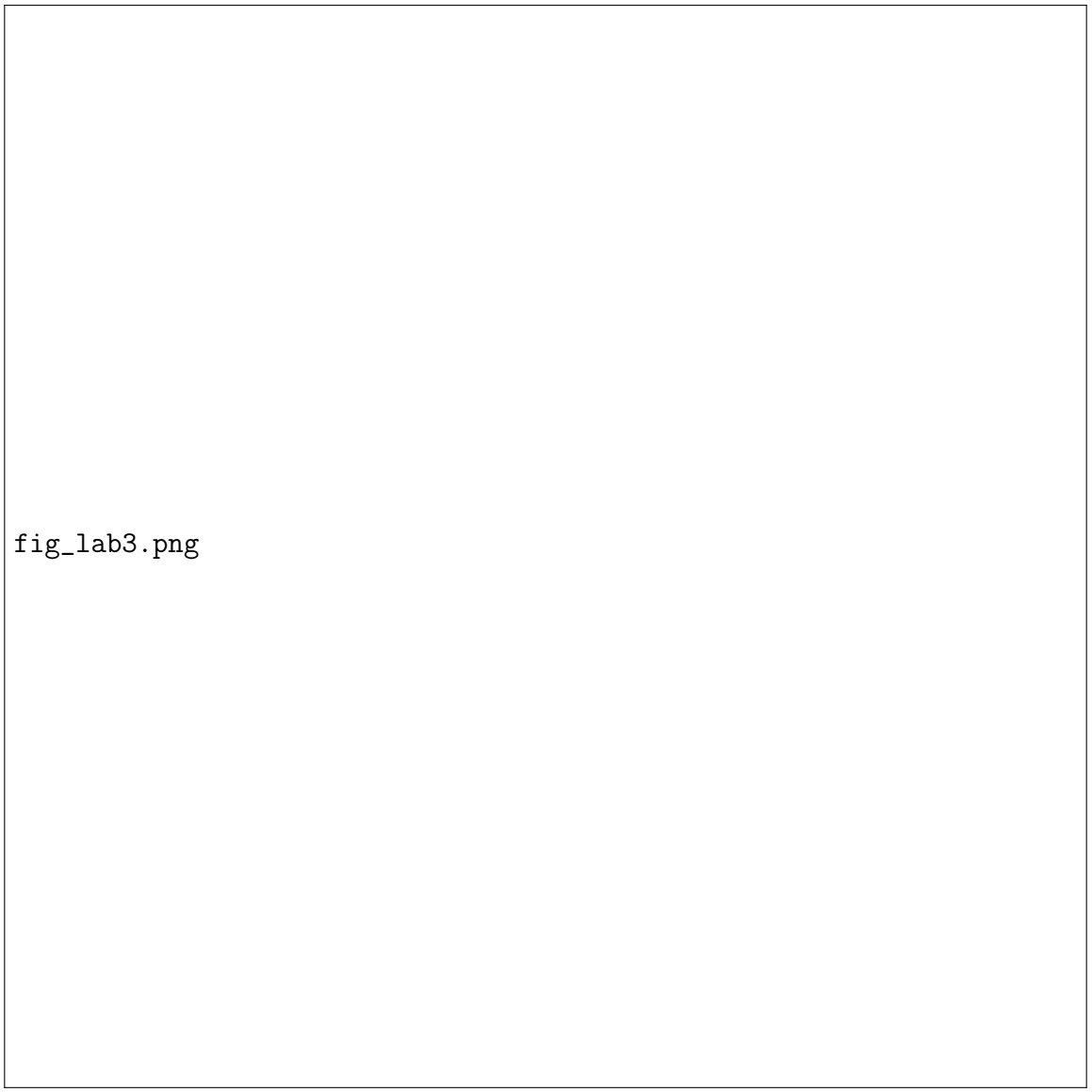


Рис. 1: Отрисовка алгоритмом By и вывод журнала операций

4.1 Преодоление ошибки переполнения в алгоритме By

При реализации алгоритма сглаживания By была обнаружена критическая ошибка `invalid color name`. Из-за погрешности вычислений с плавающей точкой (`float` в Python) дробная часть в редких случаях могла принимать значение 1.0000000000000002. При умножении на 255 для получения HEX-кода цвета это приводило к значению 256, что ломало генератор цвета. Проблема была решена введением жёсткого математического ограничения (clamping) интенсивности в диапазон $[0.0, 1.0]$.

4.2 Ограничения платформы (Windows XP)

В связи с использованием современного интерпретатора Python 3.12 и графической библиотеки CustomTkinter, нативная поддержка ОС Windows XP технически невозможна (поддержка Windows XP в Python прекращена начиная с версии 3.5). В качестве решения для современных Windows-систем был собран автономный исполня-

емый файл (`.exe`) с использованием утилиты `PyInstaller` и эмулятора `Wine` в среде Linux.

5 Временные характеристики алгоритмов

Замеры производились высокоточным таймером `time.perf_counter_ns()` исключительно на математический расчёт массива координат (рендеринг GUI был вынесен за пределы таймера). Тестовый отрезок: от $(0, 0)$ до $(1000, 1000)$.

Алгоритм	**Время (нс)**	**Анализ**
Пошаговый (Линия)	$\approx 791\,366$	Одно float-умножение и один вызов `round()` на итерацию.
ЦДА (Линия)	$\approx 1\,222\,566$	**Самый медленный в Python.** Два вызова `round()` на каждый пиксель.
Брезенхем (Линия)	$\approx 602\,093$	**Самый быстрый.** Исключительно целочисленные операции.
Сглаживание (Ву)	$\approx 1\,905\,185$	Самый ресурсоёмкий алгоритм: `float`-арифметика + два пикселя на столбец.

Таблица 2: Замеры времени математического расчёта (длина 1000 пикселей)

6 Выводы

На основе выполненной работы можно сделать следующие выводы:

1. Парадокс производительности ЦДА в Python. В классической теории (и компилируемых языках вроде C++) алгоритм ЦДА работает быстрее пошагового, так как заменяет умножение на сложение. Однако наши замеры выявили обратную картину: в интерпретируемом Python ЦДА оказался самым медленным алгоритмом ($\approx$ 1.2 млн нс против $\approx$ 791 тыс. нс). Это связано с накладными расходами виртуальной машины: в пошаговом алгоритме функция `round()` вызывается один раз на итерацию (для оси Y), а в ЦДА — дважды (для X и Y). Накладные расходы на вызовы функций в Python полностью нивелируют выгоду от замены математической операции. Это доказывает, что при выборе алгоритма необходимо учитывать особенности среды исполнения.

2. Доминирование целочисленной арифметики. Алгоритм Брезенхема является де-факто стандартом благодаря полному отказу от вещественных типов (`float`). Замена вычисления абсолютных координат на анализ знака целочисленной ошибки делает алгоритм детерминированным, абсолютно точным (отклонение всегда $\leq 1/2$ пикселя) и невероятно быстрым ($\approx$ 602 тыс. нс). Именно эта логика позволяет аппаратно реализовывать рендеринг на уровне микрокода видеокарт.

3. Практическая уязвимость вещественной арифметики (Баг переполнения цвета). Слабость вычислений с плавающей точкой была наглядно продемонстрирована при реализации сглаживания Ву. Из-за машинной погрешности стандарта IEEE 754 дробная часть координаты в редких случаях принимала значение 1.0000000000000002 вместо 1.0. При умножении этого значения на 255 (для конвертации в 8-битный HEX-формат цветового канала RGB) программа получала значение

256, что приводило к переполнению и фатальному падению графического интерфейса с ошибкой `invalid color name`. Данный баг (впоследствии устранённый жёстким математическим ограничением `max(0.0, min(1.0, value))`) чётко показал, почему в критических низкоуровневых графических системах разработчики всегда отдают предпочтение целочисленным алгоритмам Брезенхема.

4. Компромисс сглаживания (Anti-aliasing). Алгоритм Ву возвращает нас к операциям с плавающей точкой и увеличивает время расчёта более чем в 3 раза по сравнению с Брезенхемом. Однако это оправданная цена за субпиксельную точность: визуально линия становится идеально гладкой за счёт математически выверенной манипуляции прозрачностью соседних пикселей.